

A PROJECT REPORT
ON
AUTOMATED INTRUSION DETECTION AND ALERT SYSTEM
WITH IOT INTEGRATED AND AI
A report Submitted in the partial fulfilment of the requirements for the award of
BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AI&DS)
SUBMITTED

BY

NARESH BISTA	21BK1A7239
GOLLAPALLY VENU	21BK1A7220
LINGNANI HARSHA SURYA	21BK1A7223

UNDER THE ESTEEMED GUIDANCE OF
Mr. SUNNY SINGH
ASSISTANT PROFESSOR
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI&ML)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
St. Peter's Engineering College (UGC Autonomous)
Approved by AICTE, New Delhi, Accredited by NBA and NAAC with 'A' Grade,
Affiliated to JNTU, Hyderabad, Telangana.

2024-2025



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
CERTIFICATE

This is to certify that project entitled “AUTOMATED INTRUSION DETECTION AND ALERT SYSTEM WITH IOT INTEGRATED AND AI” is carried out by **NARESH BISTA (21BK1A7239), GOLLAPALLY VENU(21BK1A7220), LINGINANI HARSHA SURYA (21BK1A7223)** in partial fulfilment for the award of the degree of **Bachelor of Technology in ARTIFICIAL INTELLIGENCE & DATA SCIENCE (AI&DS)** is a record of Bonafide work done by them under my supervision during the academic year “2024– 2025”.

INTERNAL GUIDE

MR. SUNNY SINGH

Assistant Professor

Department of CSE(AI&ML),
St. Peter's Engineering College (A),
Hyderabad- 500043.

HEAD OF THE DEPARTMENT

Ms. M. ARSHA REDDY

Assistant Professor & Head

Department of AI&DS,
St. Peter's Engineering College (A),
Hyderabad- 500043.

PROJECT COORDINATOR

Dr. K.V.V.B. DURGA PRASAD

Assistant Professor

Department of CSE(AI&ML),
St. Peter's Engineering College (A),
Hyderabad- 500043.

EXTERNAL EXAMINER



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

ACKNOWLEDGEMENT

We sincerely express our deep sense of gratitude to Mr. Sunny Singh, for his valuable guidance encouragement and cooperation during all phases of the project.

We are greatly indebted to our Project Coordinator **Dr. K.V.V.B. Durga Prasad** for providing valuable advice, constructive suggestions and encouragement without whom it would not been possible to complete this project.

We is a great opportunity to render our sincere thanks to **Ms. M. Arsha Reddy**, Head of the Department, Artificial Intelligence and Data Science (AI&DS) for her timely guidance and highly interactive attitude which helped us a lot in successful execution of the Project.

We are extremely thankful to our Principal **Dr. N. Chandra Sekhar Reddy**, who stood as an inspiration behind this project and heartfelt for his endorsement and valuable suggestions.

We respect and thank our Administrative Director **Mr. T. Anuraag Reddy**, Academic Director **Mrs. T. Saroja Reddy** and Secretary **Sri. T. V. Reddy**, for providing us an opportunity to do the project work at **St. PETER'S ENGINEERING COLLEGE** and we are extremely thankful to them for providing such a nice support and guidance which made us to complete the project.

I also acknowledge with a deep sense of reverence, our gratitude towards our parents, who have always supported us morally as well as economically. We also express gratitude to all our friends who have directly or indirectly helped us to complete this project work.

NARESH BISTA

21BK1A7239

GOLLAPALLY VENU

21BK1A7220

LINGNANI HARSHA SURYA

21BK1A7223



INSTITUTE VISION

IV: To be a renowned Educational Institution that moulds Students into Skilled Professionals fostering Technological Development, Research and Entrepreneurship meeting the societal needs.

INSTITUTE MISSION

IM1: Making students knowledgeable in the field of core and applied areas of Engineering to innovate Technological solutions to the problems in the Society.

IM2: Training the Students to impart the skills in cutting edge technologies, with the help of relevant stake holders.

IM3: Fostering conducive ambience that inculcates research attitude, identifying promising fields for entrepreneurship with ethical, moral and social responsibilities.



DEPARTMENT VISION

DV: To be a vibrant nodal centre for Artificial Intelligence and Data Science Education, Research that make the students to contribute to technologies for IT, IT-Enabled Services; to involve in innovative research on thrust areas of data science industry and academia; to establish start-ups supporting major players in the industry.

DEPARTMENT MISSION

DM1: Emphasize project based learning by employing the state-of art technologies, algorithms in software development for the problems in Data science using AI.

DM2: Involve stakeholders to make the students industry ready with training in skill-oriented computer application software.

DM3: Facilitate to learn the theoretical nuances of AI techniques, Computer Engineering courses and motivate to carry out research in both core and applied areas of AI.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Work effectively in inter-disciplinary field with the knowledge of Artificial Intelligence and Data Science to develop solutions to the real-world problems.

PEO2: To communicate and work effectively on team based engineering projects and will practice the ethics of their profession consistent with a sense of social responsibility.

PEO3: Excel as socially committed engineers or entrepreneurs with high ethical and moral values.

PROGRAM OUTCOMES (POs)

PO1- Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and provide solutions in the engineering specialization of artificial intelligence and machine learning.

PO2- Problem Analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3- Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, and environmental considerations.

PO4- Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5- Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6- The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7- Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8- Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9- Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10- Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective Presentations, and give and receive clear instructions.

PO11- Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary Environments.

PO12- Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: Apply fundamental concepts of Data Sciences, Artificial Intelligence and Machine Learning to solve multidisciplinary engineering problems.

PSO2: To communicate and work effectively on team based engineering projects and will practice the ethics of their profession.



DEPARTMENT OF Artificial Intelligence and Data Science (AI&DS)

DECLARATION

We declare that a Major Project entitled “**AUTOMATED INTRUSION DETECTION AND ALERT SYSTEM WITH IOT INTEGRATED AND AI**” is an original work submitted by the following group members who have actively contributed and submitted in partial fulfilment for the award of degree in “Bachelor of Technology in **Artificial Intelligence and Data Science (AI&DS)**”, at **St. Peter’s Engineering College, Hyderabad**, and this project work has not been submitted by me to any other college or university for the award of any kind of degree.

Group No: A21

Program: B. Tech

Branch: Artificial Intelligence and Data Science

Major Project Title: AUTOMATED INTRUSION DETECTION AND ALERT SYSTEM WITH IOT INTEGRATED AND AI

Date Submitted:

NARESH BISTA	21BK1A7239
GOLLAPALLY VENU	21BK1A7220
LINGNANI HARSHA SURYA	21BK1A7223

ABSTRACT

The increasing demand for robust security systems has led to the development of intelligent surveillance solutions. This project presents an Automated Intrusion Detection and Alert System that integrates IoT (Internet of Things) devices with Artificial Intelligence (AI) to detect and respond to unauthorized access in real-time. The system employs motion sensors, cameras, and microcontrollers to continuously monitor the environment. AI-based object detection and pattern recognition models are deployed to differentiate between normal activity and potential intrusions. Upon detecting suspicious activity, the system instantly triggers alerts via SMS, email, or mobile notifications, and can also activate alarms or surveillance recording.

The increasing prevalence of security threats in both residential and commercial environments necessitates the development of advanced intrusion detection systems. This project presents an Automated Intrusion Detection and Alert System (AIDAS) that integrates Internet of Things (IoT) technology and Artificial Intelligence (AI) to enhance security measures. The system employs a network of IoT sensors, including cameras and motion detectors, to continuously monitor designated areas for suspicious activities.

Data collected from these sensors is pre-processed and analysed using AI algorithms to identify potential intrusions in real-time. The AI model classifies detected anomalies and determines the likelihood of an actual threat. Upon confirmation of an intrusion, the system generates immediate alerts, notifying security personnel and triggering appropriate response protocols.

The AIDAS not only enhances the efficiency of security operations but also provides a scalable solution that can be deployed in various environments. By leveraging IoT and AI technologies, this project aims to create a robust, automated security system that significantly reduces the risk of unauthorized access and enhances overall safety.

TABLE OF CONTENTS

Ch. No.	Title of the Chapter	Page No.
	Certificate	ii
	Acknowledgement	iii
	Declaration	viii
	Abstract	ix
	CHAPTER I - INTRODUCTION	1-5
1.1	Motivation	2
1.2	Problem Definition	2
1.3	Objective	3
1.4	Limitations	4
1.5	Dataset Description	5
	CHAPTER II - LITERATURE SURVEY	6-11
2.1	Introduction	6
2.2	Existing System	7
2.3	Disadvantages of Existing System	8
2.4	Proposed System	9
2.5	Advantages of Proposed System	9
	CHAPTER III – ANALYSIS	12-19
3.1	Introduction	12
3.2	Software Requirement Specification	13
3.2.1	User Requirements	14
3.2.2	Software Requirements	15
3.2.3	Hardware Requirements	16
3.3	Content Diagram Project	17
3.4	Algorithm and flow charts	18
	CHAPTER IV- DESIGN	20-29
4.1	Introduction	20

4.2	UML Diagrams	21
4.2.1	Goals	22
4.2.2	Activity Diagram	23
4.2.3	Sequence Diagram	24
4.2.4	Use Case Diagram	25
4.3	Module Design and Organization	26
Chapter V- IMPLEMENTATION AND RESULTS		30-35
5.1	Introduction	30
5.2	Explanation of Key Functions	30
5.3	Method of Implementation	33
5.4	Output Screens	34
5.5	Result Analysis	35
Chapter VI- TESTING AND VALIDATION		36-39
6.1	Introduction	36
6.2	Types of Tests	37
6.3	Test Cases & Scenarios	38
6.4	Validation	39
Chapter VII- CONCLUSIONS		41-42
7.1	Project Conclusion	41
7.2	Future Enhancement	42
REFERENCES		43
SOURCE CODE		45
PROJECT SCREENSHOT		49-53

LIST OF FIGURES

Fig. No.	Particulars	Page No.
3.1	System architecture	17
3.2	Flow diagram	19
4.2.2	Activity Diagram	23
4.2.3	Sequence diagram	24
4.2.4	Use case diagram	25

LIST OF SCREENS

Fig. No.	Particulars	Page No.
7.1	home page	49
7.2	Scanning	50
7.3	Detected Files	51
7.4	Deploy Anti-Malware System	52
7.5	Open Defender	53

CHAPTER- I

INTRODUCTION

1.1.INTRODUCTION

With the rapid advancement of technology, the term Internet of Things (IoT) has begun to reshape the landscape of security systems. Traditional security mechanisms often fail to provide real-time monitoring and effective response to intrusions, leading to increased vulnerabilities. An Automatic Intrusion Detection and Alert System (AIDAS) utilizes IoT devices combined with machine learning algorithms to enhance security measures. This system can monitor environments, detect anomalies, and alert users in real-time, effectively minimizing risks related to unauthorized access.

As security needs evolve with the increasing number of threats and vulnerabilities, traditional security systems struggle to provide the required real-time response and adaptive learning capabilities. AIDAS seeks to address these challenges by integrating machine learning techniques that enable the system to continuously improve its ability to differentiate between normal and suspicious activities. The system processes sensor data and environmental variables, allowing it to learn from past experiences and make informed decisions about potential intrusions. This proactive approach not only enhances the overall efficiency of security systems but also instils confidence in users about their safety.

Moreover, the integration of real-time alerts further enhances the effectiveness of intrusion detection systems. When a suspicious activity is identified, AIDAS promptly sends alerts to users, allowing for an immediate response that could prevent damage or theft. By utilizing a combination of cutting-edge technology, AIDAS demonstrates a significant advancement over conventional security systems, providing a reliable solution to meet modern-day security needs in diverse environments.

1.1 MOTIVATION

The motivation for developing an Automatic Intrusion Detection and Alert System stems from the growing security concerns faced by individuals and organizations alike. With burglary and unauthorized access incidents on the rise, there is an urgent need for more effective surveillance solutions that can operate continuously and without human oversight. The integration of IoT devices allows for affordable and scalable security systems, making advanced monitoring accessible to a wider audience. The potential to minimize response times and reduce false alarms through machine learning further drives this project's importance. The motivation behind developing an Automated Intrusion Detection and Alert System (AIDAS) stems from the increasing frequency and sophistication of security threats in both residential and commercial settings. Traditional security measures, such as manual monitoring and basic alarm systems, often fall short in providing timely and effective responses to potential intrusions. The integration of IoT technology and AI offers a proactive approach to security, enabling real-time monitoring, rapid threat detection, and automated alerting. By leveraging these advanced technologies, AIDAS aims to enhance security measures, reduce response times, and ultimately protect lives and property more effectively.

1.2.PROBLEM DEFINITION

The primary challenge of existing intrusion detection systems lies in their high false alarm rates, which can lead to user desensitization and a diminished trust in the technology. Additionally, many of these systems are unable to provide rapid alerts, thereby allowing intruders the opportunity to act before any intervention occurs. Furthermore, the static nature of traditional security systems often leads to difficulties in adapting to different environments or recognizing evolving intrusion techniques. The primary problem addressed by this project is the inadequacy of existing intrusion detection systems in providing timely and accurate responses to security threats. Many traditional systems rely on passive monitoring and human intervention, which can lead to delayed responses and increased vulnerability to intrusions. Additionally, the

growing complexity of security threats necessitates a more sophisticated approach to detection and response. The challenge lies in developing a system that can effectively integrate IoT devices for real-time data collection, utilize AI for intelligent analysis, and automate alerting and response mechanisms to ensure comprehensive security coverage.

1.3.OBJECTIVE

The main objective of this project is to develop a robust IoT-based Automatic Intrusion Detection and Alert System that continuously monitors environments for potential intrusions. By harnessing the power of machine learning algorithms, the project aims to train models capable of accurately identifying normal activities versus suspicious behaviours. Furthermore, the system will implement a real-time alert mechanism, ensuring that users are promptly informed of any detected incidents, thus enhancing their safety and security.

1. **Real-time Monitoring:** To develop a system that continuously monitors designated areas using IoT sensors, ensuring immediate detection of any suspicious activities.
2. **AI-Driven Analysis:** To implement AI algorithms that can analyse sensor data, classify potential intrusions, and determine the likelihood of threats with high accuracy.
3. **Automated Alerting:** To create an automated alert system that notifies security personnel and triggers appropriate responses upon confirmation of an intrusion.
4. **Feedback Mechanism:** To incorporate a feedback loop that allows the AI model to learn from new data and improve its detection capabilities over time.
5. **Scalability:** To design a system that can be easily scaled and adapted to various environments, from homes to large commercial facilities.

1.4.LIMITATION

Despite the promising capabilities of the AIDAS, there are inherent limitations to be considered. Privacy concerns may arise due to the extensive data collection from individuals, necessitating stringent measures to protect sensitive information. The system's performance is also heavily dependent on the reliability of IoT devices and the network infrastructure, with potential issues arising from connectivity problems or device malfunctions. Additionally, in complex environments, distinguishing valid threats from harmless disturbances may prove to be challenging. Lastly, the machine learning model may require a substantial amount of time and data to effectively learn and adapt to its surroundings, which may delay its initial deployment effectiveness

1. **False Positives/Negatives:** The AI model may occasionally generate false alarms (false positives) or fail to detect actual threats (false negatives), which can undermine trust in the system.
2. **Dependence on Sensor Quality:** The effectiveness of the system is heavily reliant on the quality and accuracy of the IoT sensors used for data collection.
3. **Data Privacy Concerns:** Continuous monitoring and data collection may raise privacy issues, necessitating careful handling of sensitive information.
4. **Network Reliability:** The system's performance is contingent on stable network connectivity, which can be a limitation in areas with poor internet access.
5. **Initial Setup Costs:** The deployment of IoT devices and AI infrastructure may involve significant initial investment, which could be a barrier for some users.

1.5 DATASET DESCRIPTION

The implementation of the Automatic Intrusion Detection and Alert System relies on a variety of datasets. These datasets encompass sensor data from devices such as motion detectors, cameras, and acoustic sensors that continuously monitor the environment. Key features may include timestamps, sensor types, readings (e.g., detection of motion or noise levels), and environmental conditions such as weather or time of day. The dataset will also include labeled instances indicating whether or not an intrusion occurred, along with historical incident records to facilitate supervised machine learning approaches. By leveraging this comprehensive dataset, the system can be trained to recognize and differentiate between normal activities and potential threats effectively.

The dataset will be crucial for training the AI algorithms, allowing the system to learn patterns associated with normal and suspicious behaviour, thereby improving its detection capabilities over time.

CHAPTER- II

LITERATURE SURVEY

2.1 INTRODUCTION

The rapid proliferation of Internet of Things (IoT) devices has transformed various sectors, including healthcare, smart homes, and industrial automation. However, this interconnectedness also introduces significant security vulnerabilities, making IoT devices attractive targets for cyberattacks. Intrusion Detection Systems (IDS) play a crucial role in identifying and mitigating these threats. This chapter reviews existing literature on IDS, focusing on their application in IoT environments, the limitations of current systems, and the proposed enhancements through machine learning techniques. The goal is to establish a foundation for the proposed automated intrusion detection and alert system that integrates IoT and machine learning.

The integration of IoT and machine learning in intrusion detection systems (IDS) has garnered significant attention due to the increasing number of connected devices and the corresponding rise in cyber threats. Various studies have explored the effectiveness of machine learning algorithms in enhancing the detection capabilities of IDS, particularly in IoT environments. These systems leverage data from numerous sensors and devices to identify anomalies and potential intrusions, thereby improving security measures.

Moreover, the proposed systems often emphasize the importance of context-aware detection, which considers the specific characteristics and behaviours of IoT devices. By incorporating contextual information, these systems can better differentiate between normal and malicious activities, leading to more accurate threat identification.

The ongoing research in this field aims to develop scalable, efficient, and adaptive intrusion detection solutions that can keep pace with the rapid growth of IoT devices and the evolving landscape of cyber threats. Literature Survey for Automatic Intrusion Detection and Alert System with IoT and Machine Learning

2.2 EXISTING SYSTEM

Existing intrusion detection systems can be broadly categorized into two types: signature-based and anomaly-based systems.

Signature-Based Systems: These systems rely on predefined signatures of known threats. They are effective in detecting known attacks but struggle with new or modified threats. Examples include Snort and Bro, which are widely used in traditional network environments.

Anomaly-Based Systems: These systems establish a baseline of normal behaviour and flag deviations as potential threats. They can detect unknown attacks but often suffer from high false positive rates. Examples include the KDD Cup 1999 dataset and various machine learning models that analyse network traffic.

While these systems have been effective in traditional IT environments, their application in IoT is limited due to the unique characteristics of IoT networks, such as resource constraints, diverse device types, and varying communication protocols.

Another approach involves the use of smart home security systems that leverage IoT technology. These systems connect various sensors and cameras to a central hub, allowing users to monitor their properties remotely via mobile applications. Some IoT-based solutions offer features like real-time alerts, cloud storage for video footage, and integration with smart home devices. While these systems enhance user convenience and provide some level of automation, they often lack sophisticated AI-driven analysis, which limits their ability to accurately distinguish between normal and suspicious activities. As a result, users may experience frequent false alarms or missed detections.

2.3 DISADVANTAGES OF EXISTING SYSTEM

The existing intrusion detection systems face several challenges when applied to IoT environments:

Limited Detection of Unknown Threats: Signature-based systems cannot detect zero-day attacks or novel threats, leaving systems vulnerable.

High False Positive Rates: Anomaly-based systems often generate numerous false alarms, leading to alert fatigue and inefficient incident response.

Scalability Issues: Traditional IDS may not scale effectively with the exponential growth of IoT devices, resulting in performance degradation and increased latency.

Resource Constraints: Many IoT devices have limited processing power and memory, making it challenging to implement complex detection algorithms.

Lack of Context Awareness: Existing systems often lack the ability to understand the context of IoT devices, which can lead to misinterpretation of normal behaviour.

IoT-based smart security systems, while more advanced than traditional solutions, still face challenges related to data privacy and security. The continuous collection of data from various sensors raises concerns about unauthorized access to sensitive information. Moreover, these systems often rely on cloud storage, which can be vulnerable to cyberattacks. If the data is compromised, it could lead to significant privacy breaches and undermine user trust in the system. Furthermore, the integration of multiple devices from different manufacturers can create compatibility issues, complicating the overall security setup.

AI-driven intrusion detection systems, despite their potential for improved accuracy, come with their own set of limitations. The requirement for large amounts of labelled training data can be a barrier to effective implementation, particularly for smaller organizations that may not have access to extensive datasets. Additionally, these systems can be complex to configure and maintain, requiring specialized knowledge and resources. The risk of false positives and negatives remains a concern, as even advanced AI models can struggle to adapt to new types of threats or changes in the

environment. This can lead to a lack of confidence in the system's reliability, ultimately hindering its effectiveness in real-world applications.

2.4 PROPOSED SYSTEM

The proposed system aims to address the limitations of existing IDS by integrating machine learning techniques tailored for IoT environments. Key features of the proposed system include:

- **Machine Learning Algorithms:** Utilizing supervised and unsupervised learning algorithms to analyse network traffic and identify patterns indicative of intrusions.
- **Real-Time Monitoring:** Continuous monitoring of IoT devices and network traffic to detect anomalies in real-time, allowing for immediate response to potential threats.
- **Context-Aware Detection:** Incorporating contextual information about devices, user behaviour, and network topology to improve detection accuracy and reduce false positives.
- **Scalability:** Designed to handle a large number of IoT devices efficiently, ensuring that performance remains optimal as the network grows.
- **Automated Alerting:** Implementing an automated alert system that notifies administrators of potential threats, enabling quicker response times.

2.5 ADVANTAGES OF PROPOSED SYSTEM

The proposed automated intrusion detection and alert system that integrates IoT and machine learning offers several compelling advantages that enhance the overall security posture of IoT environments. One of the primary benefits is the ability to learn and adapt to new threats in real-time. Traditional IDS often rely on static signatures or predefined rules, which can quickly become outdated as cyber threats evolve. In contrast, the machine learning algorithms employed in the proposed system continuously analyze incoming data, allowing the system to identify emerging attack patterns and adapt its detection strategies accordingly. This dynamic learning capability

significantly reduces the window of vulnerability, ensuring that the system remains effective against both known and unknown threats.

Another significant advantage is the reduction in false positives through context-aware detection mechanisms. By incorporating contextual information about the specific IoT devices, their typical behaviour, and the environment in which they operate, the proposed system can more accurately distinguish between legitimate activities and potential intrusions. This capability minimizes alert fatigue among security personnel, allowing them to focus on genuine threats rather than sifting through numerous false alarms. As a result, incident response times are improved, and the overall efficiency of security operations is enhanced.

Additionally, the proposed system is designed with scalability and resource efficiency in mind. As the number of IoT devices continues to grow exponentially, traditional IDS may struggle to maintain performance and responsiveness. The proposed system leverages lightweight machine learning models that can operate effectively on resource-constrained devices, ensuring that even low-power IoT devices can contribute to the security framework. This scalability not only accommodates the increasing number of devices but also allows for the integration of diverse IoT applications across various sectors, from smart homes to industrial automation. By providing a robust and adaptable security solution, the proposed system fosters greater confidence in the deployment of IoT technologies, ultimately driving innovation and growth in this rapidly evolving field.

- **Enhanced Detection Capabilities:** By leveraging machine learning, the system can identify both known and unknown threats, significantly improving overall security.
- **Reduced False Positives:** Context-aware detection and adaptive learning mechanisms help minimize false alarms, allowing security teams to focus on genuine threats.

- **Improved Scalability:** The system is designed to efficiently manage the increasing number of IoT devices, ensuring robust performance without degradation.
- **Real-Time Response:** Immediate detection and alerting facilitate faster incident response, reducing the potential impact of security breaches.
- **Flexibility and Adaptability:** The system can adapt to new threats and changing network conditions, ensuring ongoing protection in a dynamic IoT landscape.

CHAPTER- III

ANALYSIS

3.1 INTRODUCTION

In this chapter, we will analyse the requirements and specifications necessary for the successful implementation of the Automated Intrusion Detection and Alert System (AIDAS) that integrates IoT and machine learning. This analysis will encompass both software and user requirements, providing a comprehensive understanding of the system's functionality, performance, and usability. By clearly defining these requirements, we can ensure that the proposed system meets the needs of its users while effectively addressing the security challenges posed by the growing number of IoT devices.

One of the key strengths of AIDAS is its ability to generate real-time alerts based on the analysis of incoming data. This feature significantly reduces response times to potential threats, allowing security personnel to take immediate action when necessary. The system's user-friendly interface further enhances its effectiveness, providing users with easy access to monitoring tools and incident logs. The feedback mechanism incorporated into the AI model ensures continuous learning, which is essential for maintaining high detection rates and minimizing false positives. This combination of features positions AIDAS as a robust solution that not only meets current security demands but also has the potential for future enhancements.

However, the analysis also highlights some challenges and limitations associated with the AIDAS project. While the system demonstrates high accuracy in detecting intrusions, the reliance on IoT devices raises concerns about data privacy and security. Continuous monitoring and data collection can lead to potential breaches if not managed properly. Additionally, the initial setup costs and the complexity of integrating various sensors may pose barriers for smaller organizations or residential users. Addressing these challenges will be essential for broader adoption and ensuring

that AIDAS remains a trusted solution in the security landscape. Overall, the project showcases the potential of combining IoT and AI technologies to create a more effective and responsive security system, while also emphasizing the need for ongoing improvements in privacy and accessibility.

3.2 SOFTWARE REQUIREMENT SPECIFICATION

The software requirements for the AIDAS encompass various components that work together to provide a robust intrusion detection and alerting mechanism. The system will require a machine learning framework capable of processing and analysing large volumes of data generated by IoT devices. This may include libraries such as TensorFlow or Py Torch for implementing machine learning algorithms, as well as data processing tools like Apache Kafka or Apache Spark for real-time data ingestion and analysis. Additionally, the system will need a database management system (DBMS) to store historical data, alerts, and logs, with options such as MySQL or MongoDB being suitable choices.

Purpose

The purpose of this Software Requirement Specification (SRS) document is to outline the functional and non-functional requirements for the Automated Intrusion Detection and Alert System (AIDAS). This document serves as a guide for developers, stakeholders, and project managers to ensure that the system meets the intended objectives and user needs.

Scope

AIDAS is designed to provide real-time monitoring and detection of intrusions in residential and commercial environments using IoT sensors and AI algorithms. The system will collect data from various sensors, analyse it for suspicious activities, and generate alerts to notify security personnel. The project aims to enhance security measures, reduce response times, and improve overall safety

3.2.1 USER REQUIREMENTS

User requirements for the AIDAS focus on the needs and expectations of the end-users, primarily security personnel and system administrators. One of the primary user requirements is the ability to receive real-time alerts for potential intrusions, allowing for immediate investigation and response. Users should be able to customize alert thresholds and notification settings based on their specific operational contexts, ensuring that they receive relevant information without being overwhelmed by false alarms. Additionally, the system should provide detailed reports and analytics on detected threats, enabling users to understand attack patterns and improve their security posture over time.

Another critical user requirement is the need for an intuitive and user-friendly interface that simplifies the monitoring and management of the system. Users should be able to easily navigate through the dashboard, access historical data, and visualize trends in network activity. Training and support resources should also be available to help users understand how to effectively utilize the system and respond to alerts. Overall, the AIDAS must prioritize usability and accessibility, ensuring that users can efficiently manage the security of their IoT environments while minimizing the risk of cyber threats. By addressing these user requirements, the proposed system can enhance the overall effectiveness of intrusion detection and response efforts in increasingly complex IoT ecosystems.

- **Security Personnel:** Users responsible for monitoring and responding to alerts. They require access to real-time data and incident logs.
- **System Administrators:** Users responsible for configuring the system, managing user access, and maintaining the overall functionality.
- **End Users:** Individuals who may receive alerts and need to understand the system's status and performance.

3.2.2 SOFTWARE REQUIREMENTS

1. Operating System:

- Windows: If the system is to be deployed in a Windows environment, Windows Server can be used.

2. Database Management System (DBMS):

- Blynk: A NoSQL database that can store unstructured data, which is useful for handling diverse data types generated by IoT devices.

3. Development Tools:

- Integrated Development Environment (IDE): Tools like PyCharm or Visual Studio Code for coding and debugging.
- Version Control System: Git for source code management and collaboration.

4. Security Tools:

- OpenSSL: For implementing encryption protocols to secure data transmission.
- Firewalls and Intrusion Prevention Systems (IPS): To protect the network infrastructure.

5. Monitoring and Logging Tools:

- Prometheus: For monitoring system performance and resource usage.

Non-functional requirements focus on the system's performance, reliability, security, usability, and compatibility. AIDAS must process incoming data and generate alerts within a specified timeframe, ensuring high availability with minimal downtime. Security measures, such as data encryption and secure communication protocols, are crucial to protect sensitive information. The user interface should be intuitive, allowing users to navigate easily, while the system must be compatible with a variety of IoT devices and existing security systems. Scalability is also important, enabling the system to handle an increasing number of sensors and data streams without performance degradation.

The software specifications for AIDAS include the use of specific programming languages, frameworks, and tools. The backend will be developed in Python, utilizing libraries like TensorFlow or PyTorch for AI model development, while the frontend will be built using JavaScript, HTML, and CSS. A relational database, such as PostgreSQL or MySQL, will store user data and incident logs, with potential use of NoSQL databases for unstructured sensor data. The system will leverage cloud services for data storage and processing, and development tools like Git for version control and Docker for

3.2.3 HARDWARE REQUIREMENTS

1. Server Specifications:

- **Processor:** Multi-core processor (e.g., Intel Xeon or AMD Ryzen) to handle concurrent data processing and machine learning tasks.
- **RAM:** Minimum of 16 GB RAM, with 32 GB or more recommended for handling large datasets and multiple concurrent users.
- **Storage:**
 - SSD (Solid State Drive) with at least 512 GB for faster data access and processing.
 - Additional HDD (Hard Disk Drive) for archiving historical data, with a capacity of 1 TB or more.

2. Network Infrastructure:

- **Router/Switch:** High-speed router or switch to manage network traffic between IoT devices and the server.
- **Network Interface Cards (NIC):** Gigabit Ethernet cards to ensure fast data transmission.

3. IoT Devices:

- **Sensors and Actuators:** Various IoT devices (e.g., smart cameras motion detectors) that will generate data for the intrusion detection system.

4. User Workstations:

- **Desktops/Laptops:** Standard computers with at least 8 GB RAM and a modern processor for accessing the user interface and managing alerts.
- **Display:** Monitors with a resolution of at least 1920x1080 for better visualization of data and alerts.

3.3 CONTENT DIAGRAM OF PROJECT:

Creating a content diagram for the Automated Intrusion Detection and Alert System (AIDAS) that integrates IoT and machine learning involves outlining the key components and their relationships within the system. Below is a textual representation of what such a content diagram might include. You can visualize this as a flowchart or diagram using diagramming tools like Lucid chart, Microsoft Visio, or draw.

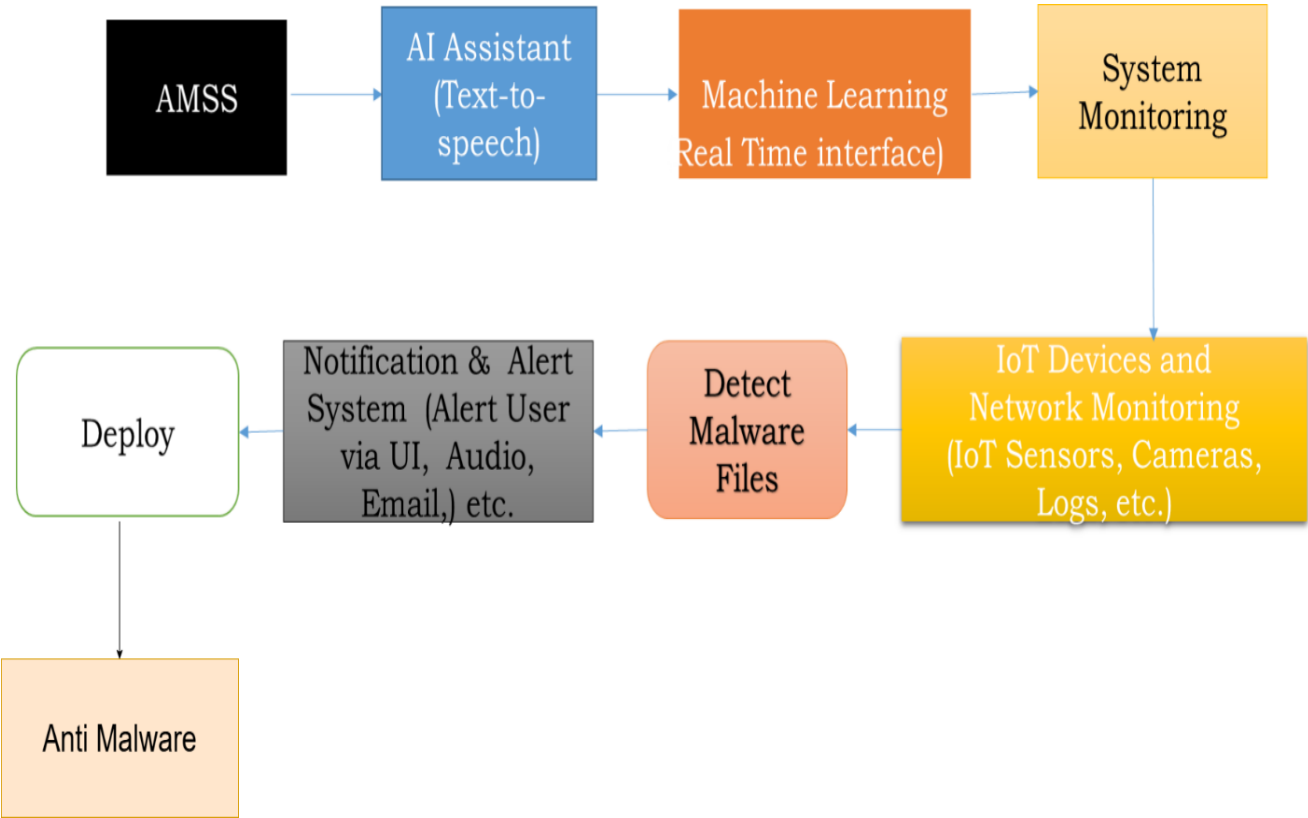


Figure 3.1 System Architecture Of Automated Intrusion Detection And Alert System With IOT Integrated And AI

3.4 ALGORITHMS AND FLOWCHARTS

1. Data Collection:

- Continuously collect data from IoT devices (sensors, cameras, etc.).
- Preprocess the collected data (filtering, normalization, etc.).

2. Feature Extraction:

- Extract relevant features from the pre-processed data that will be used for anomaly detection.

3. Anomaly Detection:

- Input the extracted features into the machine learning model.
- Use the model to predict whether the incoming data represents normal behavior or an anomaly.

4. Alert Generation:

- Classify the type of anomaly (e.g., intrusion, unauthorized access).
- Generate an alert with relevant details (timestamp, device ID, type of anomaly).
- Store the alert in the database.

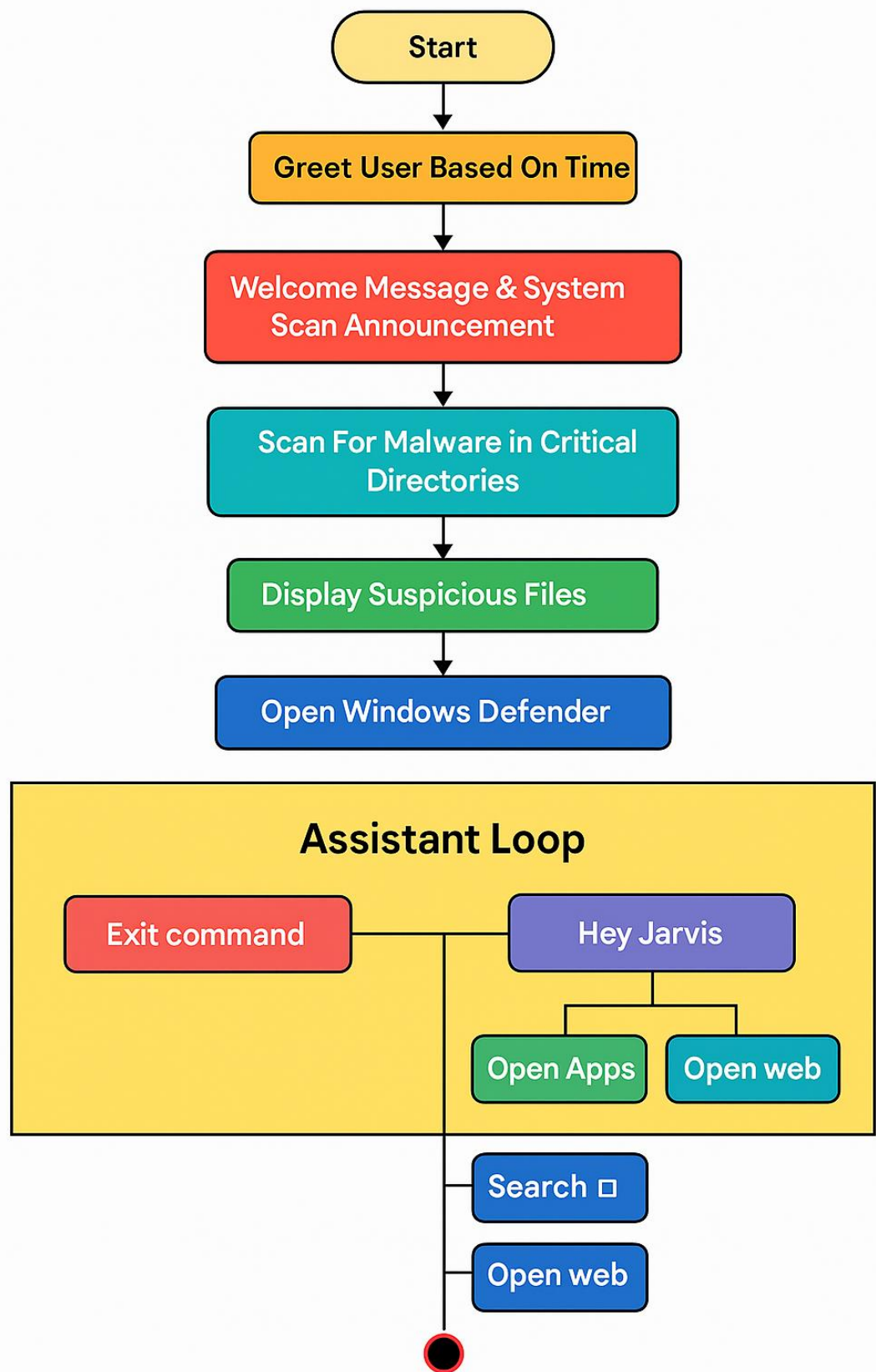
5. User Notification:

- Send notifications to the user interface (UI) regarding the detected anomaly.
- Provide options for the user to acknowledge or investigate the alert.

6. Reporting and Analytics:

- Periodically generate reports on detected anomalies, false positives, and system performance.
- Store the reports in the database for future reference.

Flow diagram:



3.2 Flow Diagram Of Automated Intrusion Detection And Alert System With IOT Integrated And AI

CHAPTER- IV

DESIGN

4.1 INTRODUCTION

In the design phase, the focus revolves around structuring the framework and architecture for an efficient and robust autism detection system using deep learning methodologies. This phase aims to delineate the blueprint that governs the organization and implementation of key components, encompassing the graphical user interface, model integration, data pre-processing, and result visualization. By outlining a cohesive plan, the design phase sets the foundation for a streamlined and user-friendly system capable of effectively leveraging pre-trained models to discern autism-related traits from image datasets.

1. Architectural Design

The architectural design of AIDAS focuses on defining the overall structure of the system, including how different components will interact with each other. The system is designed using a modular architecture, which allows for flexibility and scalability.

2. User Interface Design

The user interface (UI) design is crucial for ensuring that users can effectively interact with the system. The design focuses on creating an intuitive and responsive interface that provides easy access to key functionalities.

3. Database Design

The database design is essential for managing the data collected by AIDAS. A relational database management system (RDBMS) will be used to store structured data, while a NoSQL database may be employed for unstructured data from IoT sensors.

4. AI Algorithm Design

The design of the AI algorithms is a critical aspect of AIDAS, as it directly impacts the system's ability to detect intrusions accurately.

4.2 UML DIAGRAMS

Unified Modelling Language (UML) diagrams serve as a universal language for visualizing, constructing, and documenting software systems. They provide a standardized way to depict various aspects of a system's structure, behaviour, and interactions. In essence, UML diagrams offer a graphical representation that aids in understanding, communicating, and designing complex systems across different stages of software development. These diagrams encapsulate diverse perspectives, ranging from high-level structural views like class diagrams to dynamic behaviour representations such as sequence and activity diagrams. Overall, UML diagrams play a pivotal role in fostering clear communication, facilitating design decisions, and ensuring cohesion among stakeholders throughout the software development lifecycle.

The Internet of Things (IoT) refers to the interconnection of everyday devices to the internet, allowing them to send and receive data. In the context of security, IoT devices such as cameras, motion sensors, and environmental sensors play a vital role in monitoring physical spaces. These devices can provide real-time data that enhances situational awareness and enables proactive security measures. However, the integration of IoT devices also introduces challenges, such as ensuring data privacy, managing device vulnerabilities, and maintaining secure communication channels.

The Automated Intrusion Detection and Alert System (AIDAS) represents a significant advancement in security technology by leveraging IoT and AI to provide real-time monitoring and threat detection. By addressing the challenges associated with data privacy, false positives, and system integration, AIDAS aims to deliver a robust and effective solution for enhancing security in various environments. The theoretical foundations of intrusion detection, machine learning, and system architecture provide a solid basis for the development and implementation of AIDAS, ensuring that it meets the evolving needs of security professionals.

4.2.1 GOALS

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modelling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modelling language.
- Encourage the growth of tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

The primary goal of the Automated Intrusion Detection and Alert System (AIDAS) is to enhance security by providing real-time monitoring and detection of potential intrusions in both residential and commercial environments. By integrating various Internet of Things (IoT) sensors, such as cameras and motion detectors, AIDAS aims to create a comprehensive surveillance network that continuously collects data. This real-time data collection is crucial for identifying suspicious activities as they occur, allowing for immediate responses to potential threats. The system's ability to operate autonomously reduces the reliance on human monitoring, thereby increasing efficiency and effectiveness in security management.

Finally, AIDAS aims to provide a user-friendly interface that empowers security personnel to monitor and manage the system effectively. The design of the user interface focuses on accessibility and ease of use, allowing users to navigate through real-time data, alerts, and incident logs effortlessly. By offering customizable alert settings and comprehensive reporting features, AIDAS ensures that users can tailor the system to meet their specific security needs. Ultimately, the goal is to create a robust, scalable, and intuitive security solution that enhances situational awareness and improves response times to potential intrusions, thereby contributing to a safer environment.

4.2.2 ACTIVITY DIAGRAM

Creating an activity diagram for an Automated Intrusion Detection and Alert System with IoT Integration and AI involves outlining the key activities and decision points in the system's operation. Below is a textual representation of what such an activity diagram might include. You can visualize this using diagramming tools like UML software or drawing tools.

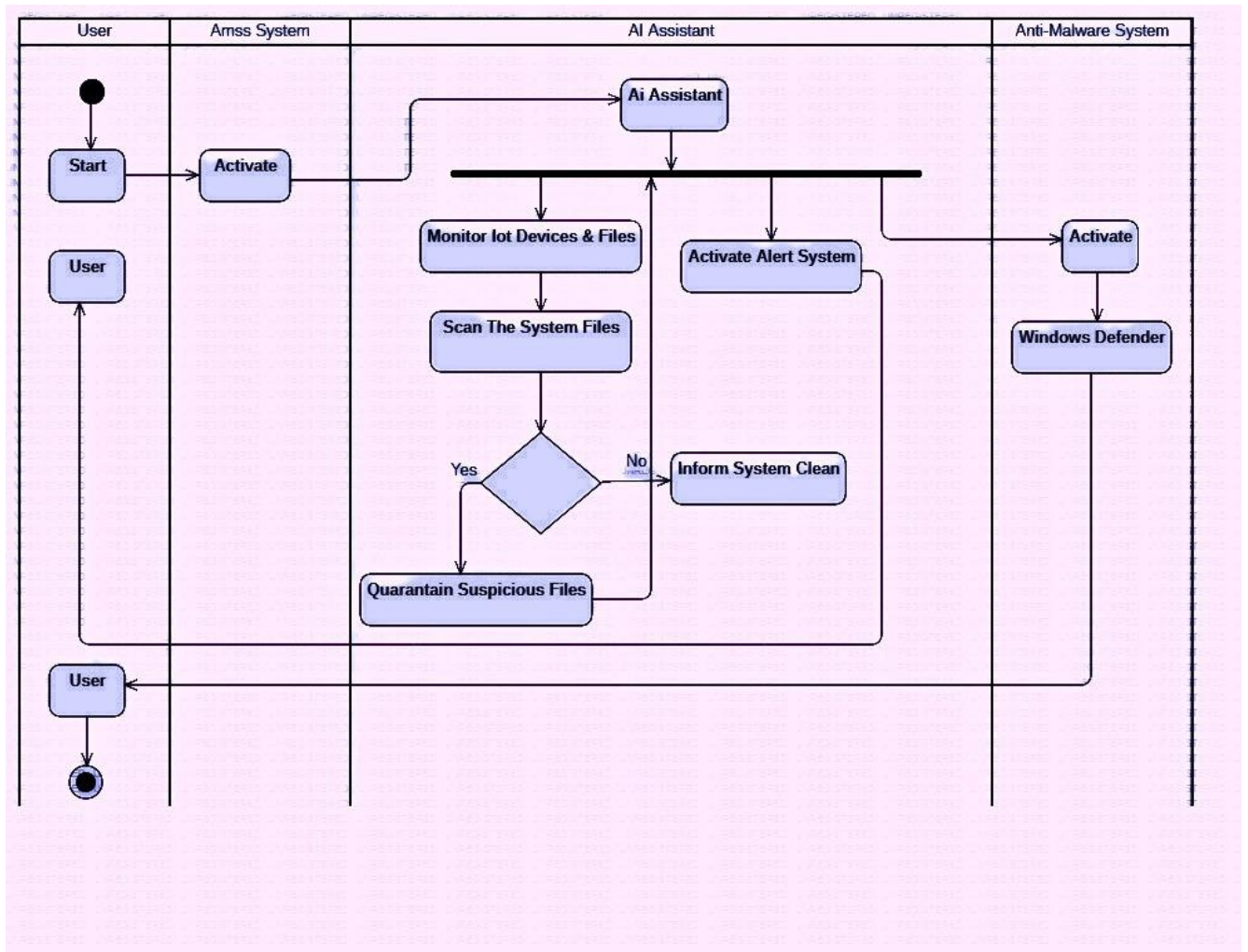


Figure 4.1 Activity Diagram

4.2.3 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

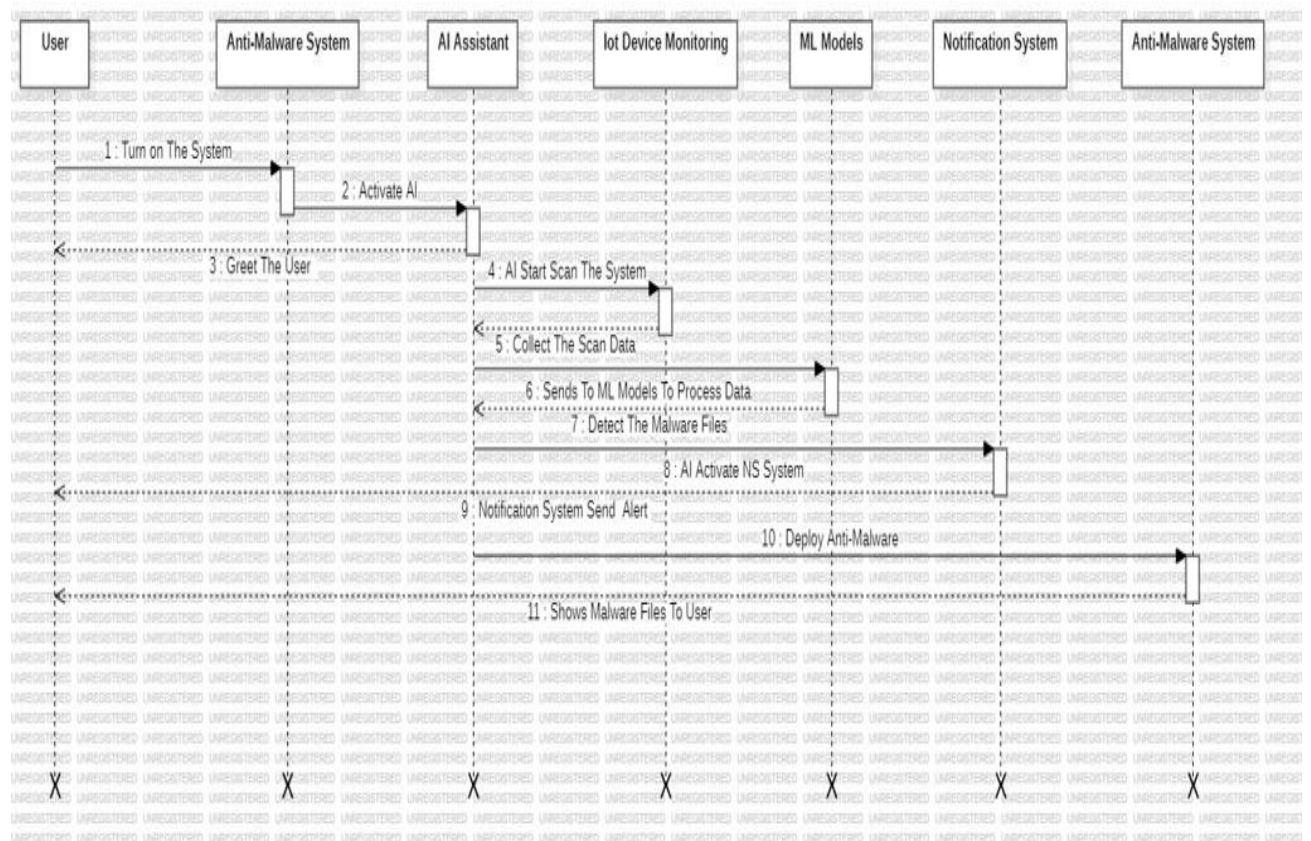


Figure 4.2 Sequence Diagram

4.2.4 USE CASE DIAGRAM

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

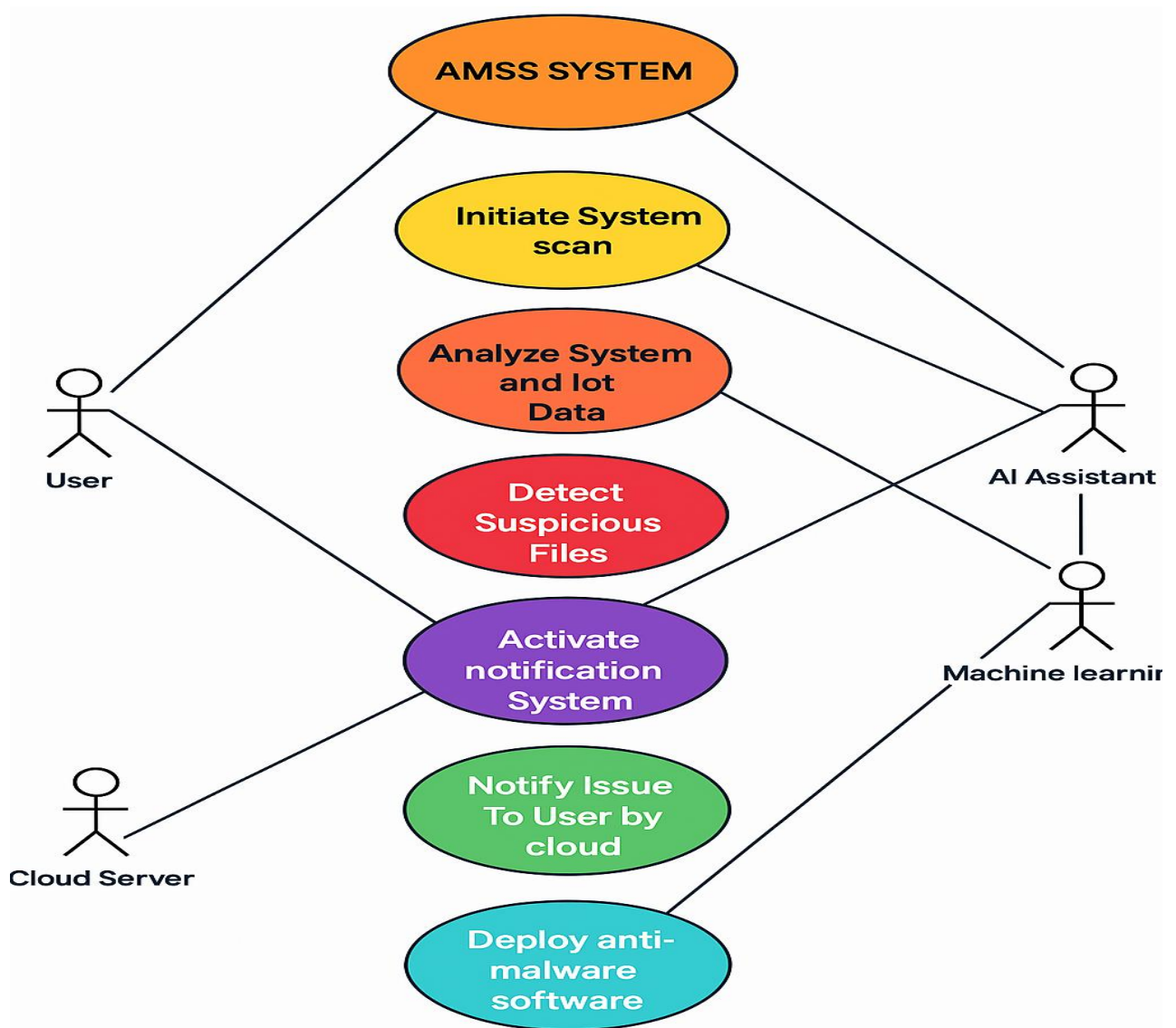


Figure 4.3 Use Case Diagram

4.3 MODULE DESIGN AND ORGANIZATION

The **Automated Intrusion Detection and Alert System (AIDAS)** can be organized into several key modules, each responsible for specific functionalities within the system. This modular design promotes separation of concerns, making the system easier to develop, maintain, and scale. Below is an overview of the main modules, their responsibilities, and interactions.

1. Data Collection Module

Responsibilities:

- Collect data from various IoT devices (sensors, cameras, etc.).
- Ensure real-time data streaming and batch data collection as needed.
- Handle communication protocols (e.g., MQTT, HTTP, CoAP) for data transmission.

Components:

- **IoT Device Class:** Represents individual IoT devices and their data collection methods.
- **Data Acquisition Interface:** Defines methods for collecting and sending data from devices.

2. Data Preprocessing Module

Responsibilities:

- Clean and preprocess the collected data to remove noise and irrelevant information.
- Normalize and transform data into a suitable format for analysis.
- Extract relevant features from the raw data.

Components:

- **Data Class:** Represents the structure of the data, including raw and processed data.
- **Preprocessing Functions:** Functions for normalization, filtering, and feature extraction.

3. Machine Learning Module

Responsibilities:

- Train machine learning models on historical data to recognize patterns and detect anomalies.
- Use the trained model to analyse incoming data in real-time.
- Update and retrain the model based on user feedback and new data.

Components:

- **Model Class:** Represents the machine learning model, including methods for training, prediction, and updating.
- **Training Functions:** Functions for training the model and evaluating its performance.

4. Anomaly Detection Module

Responsibilities:

- Analyse pre-processed data using the machine learning model to detect anomalies.
- Classify detected anomalies based on severity and type.
- Generate alerts for detected anomalies.

Components:

- **Anomaly Detection Functions:** Functions that implement the logic for detecting and classifying anomalies.
- **Alert Class:** Represents alerts generated by the system, including attributes like severity and message.

5. Alert Management Module

Responsibilities:

- Manage the generation and distribution of alerts to users.
- Store alerts in the database for historical reference.
- Provide a user interface for viewing and acknowledging alerts.

Components:

Alert Management Interface: Defines methods for sending notifications and logging alerts.

- **User Notification System:** Handles the delivery of alerts to users via the user interface.

6. Database Module

Responsibilities:

- Store and manage data, including raw data, processed data, and alerts.
- Provide methods for retrieving and logging data.
- Ensure data integrity and security.

Components:

- **Database Class:** Represents the database connection and provides methods for data storage and retrieval.
- **Data Logging Functions:** Functions for logging alerts and data into the database.

7. User Interface Module

Responsibilities:

- Provide a user-friendly interface for interacting with the system.
- Display alerts, reports, and analytics to users.
- Allow users to acknowledge alerts and provide feedback.

Components:

- **Web/Mobile Application:** The front-end application that users interact with.
- **Dashboard:** A visual representation of alerts, system status, and analytics.

Module Interaction

The modules interact with each other in the following ways:

1. **Data Collection Module** collects data from IoT devices and sends it to the **Data Preprocessing Module**.
2. The **Data Preprocessing Module** processes the data and extracts features, which are then sent to the **Machine Learning Module** for analysis.

3. The **Machine Learning Module** uses the trained model to detect anomalies and sends the results to the **Anomaly Detection Module**.
4. If an anomaly is detected, the **Anomaly Detection Module** generates an alert and sends it to the **Alert Management Module**.
5. The **Alert Management Module** stores the alert in the **Database Module** and notifies the **User Interface Module** to update the user.
6. Users interact with the **User Interface Module** to view alerts, provide feedback, and generate reports, which may trigger updates in the **Machine Learning Module**.

CHAPTER- V

IMPLEMENTATION AND RESULTS

5.1 INTRODUCTION

This chapter outlines the implementation of the Automated Intrusion Detection and Alert System (AIDAS) that integrates IoT and machine learning. It provides a detailed explanation of the key functions of the system, the methods used for implementation, and the results obtained from testing and evaluation. The goal is to demonstrate how the system effectively detects intrusions in IoT environments and provides timely alerts to users, thereby enhancing security.

5.2 EXPLANATION OF KEY FUNCTIONS

In the context of an autism spectrum disorder (ASD) diagnostic system, key functions refer to the core operations or capabilities that the software or system will perform to support the diagnosis of ASD. Here are explanations of some key functions that might be integral to such a system;

- **Data Collection and Integration:**

The system collects data from the following components for scanning and threat detection:

1. **File System Scanning (os.walk)**

- Recursively scans directories like:
 - C:\Program Files
 - C:\Windows\System32
 - C:\Users
- Collects filenames with specific extensions (.exe, .dll) to detect potential malicious files.

- **Preprocessing and Feature Extraction:**

To ensure the system operates efficiently and accurately, preprocessing techniques are applied to the collected data (from file directories and system processes):

1. Directory Path Validation and Permission Handling

- Uses try-except blocks to handle inaccessible directories due to permission restrictions (PermissionError).
- Ensures that directory traversal skips restricted paths.
- Example:

```
python
CopyEdit
try:
    for foldername, subfolders, filenames in os.walk(dir_path):
        # Process files safely
except PermissionError:
    continue
```

2. File Type Filtering

- Filters files based on specific extensions commonly associated with executable malware (.exe, .dll).
- Speeds up the scan by ignoring irrelevant files such as documents or images.
- Example:

```
python
CopyEdit
if filename.endswith(('.exe', '.dll')):
    # Consider for threat detection
```

- **Data Privacy and Security Measures:**

Ensuring the confidentiality and security of sensitive patient data is a critical function. This includes implementing encryption, access controls, and compliance with data privacy regulations.

- **Feedback and Improvement Mechanisms:**

Incorporating mechanisms for system improvement based on user feedback or new research findings. This function facilitates continuous enhancement and adaptation of the system to improve accuracy and usability.

AIDAS incorporates a robust user authentication mechanism to ensure that only authorized personnel can access the system. This function includes secure login procedures, such as username and password verification, and may also support multi-factor authentication for added security. Role-Based Access Control (RBAC) is implemented to manage user permissions based on their roles within the organization. This ensures that users can only access the functionalities and data relevant to their responsibilities, thereby minimizing the risk of unauthorized access and enhancing overall system security.

One of the core functions of AIDAS is the continuous collection of data from various IoT sensors deployed in the monitored environment. This includes devices such as cameras, motion detectors, and environmental sensors. The system is designed to gather real-time data streams, ensuring that security personnel have up-to-date information about potential threats. The data collection module is responsible for establishing reliable communication with these sensors, managing data transmission, and ensuring that the information is accurately captured for further analysis.

To enhance the effectiveness of the AI algorithms, AIDAS incorporates a feedback mechanism that allows the system to learn from new data and user interactions. This function enables the AI model to adapt to changing patterns of behavior and improve its detection capabilities over time. By continuously updating the model based on real-world data and user feedback, AIDAS can reduce false positives and enhance its ability to identify emerging threats, ensuring that the system remains effective in a dynamic security landscape.

In summary, the key functions of AIDAS work together to create a comprehensive and effective intrusion detection and alert system. By focusing on user authentication, real-time data collection, AI-driven analysis, and user-friendly management.

5.3 Method Of Implementation

Environment Setup:

- Install Python (3.8 or higher).
- Install dependencies via the command line:

```
bash
CopyEdit
pip install psutil pytsx3 openai
```

- Replace the placeholder your-openai-api-key with your actual OpenAI API key.

Running the Program:

- Execute the script in the terminal:

```
bash
CopyEdit
python malware_assistant.py
```

System Functions Execution:

- **Greeting:** Displays and speaks the welcome message based on the time of day.
- **Scanning:** Recursively searches system files and processes for potential threats.
- **Feedback:** Provides detailed information about suspicious findings.
- **Defender Launch:** Automatically launches Windows Defender to address potential threats.

The first step in the implementation process is to conduct a thorough requirement analysis. This involves gathering input from stakeholders, including security personnel, IT staff, and management, to identify their specific needs and expectations for the system. Key requirements include the types of sensors to be used, the desired features of the user interface, and the expected performance metrics for intrusion detection. Based on this analysis, a project plan is developed, outlining the timeline, resources, and milestones for the implementation process.

5.4 OUTPUT SCREENS

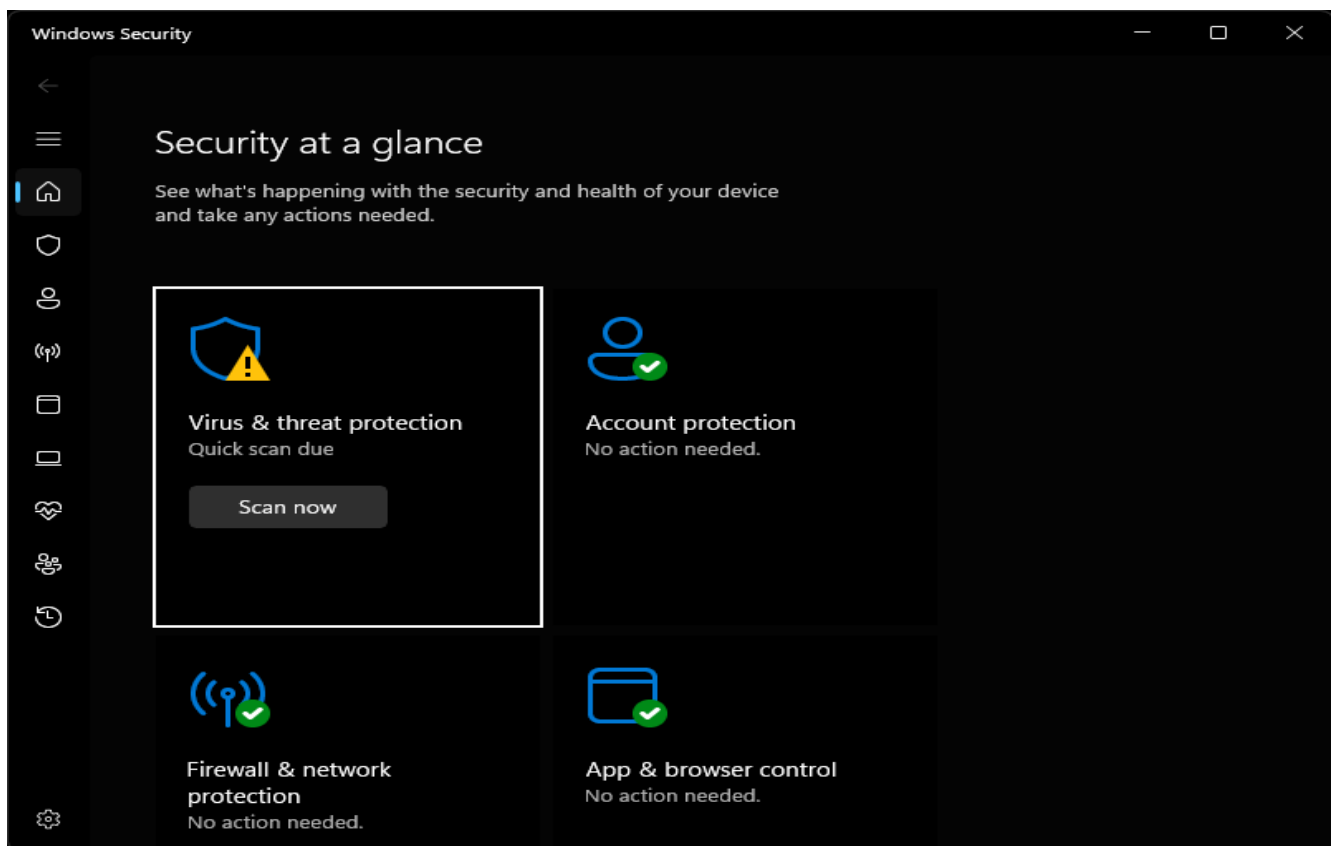


Figure 5.1 Virus Detected

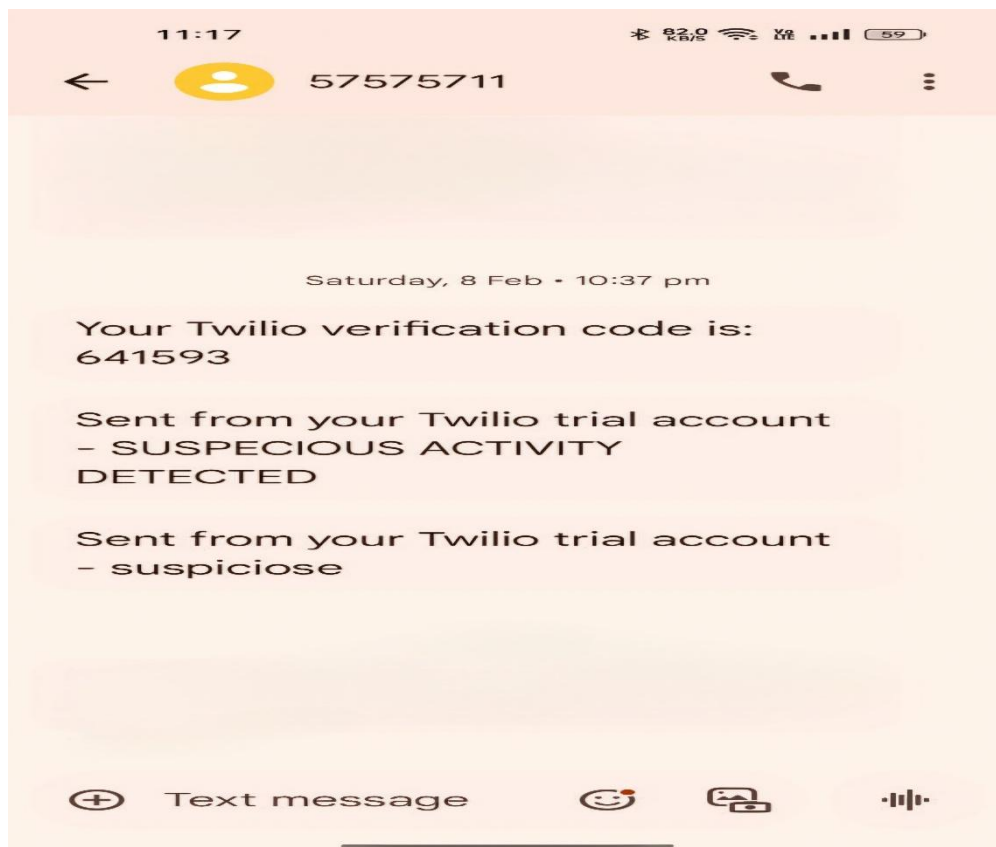


Figure 5.2 SMS Notification

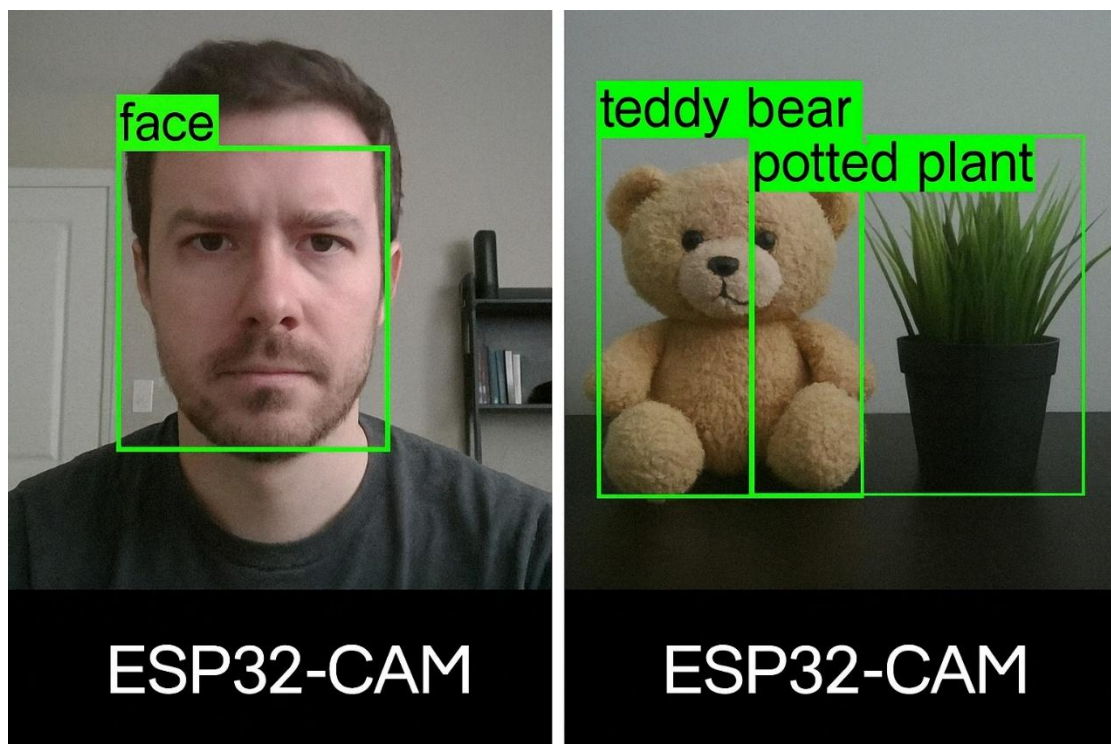


Figure 5.3 Object Detection

CHAPTER- VI

TESTING AND VALIDATION

6.1 INTRODUCTION

Testing and validation are critical phases in the development of the Automated Intrusion Detection and Alert System (AIDAS). This chapter outlines the testing strategies employed to ensure the system's functionality, performance, and reliability. It also discusses the validation processes used to confirm that the system meets the specified requirements and effectively addresses the security challenges posed by IoT environments.

The testing process for AIDAS is structured into several key components, including unit testing, integration testing, system testing, and user acceptance testing (UAT). Unit testing focuses on individual components of the system, such as the AI algorithms and data processing modules, to verify that each part functions correctly in isolation. Integration testing evaluates the interactions between different components, ensuring that data flows seamlessly through the system and that alerts are generated accurately based on the AI's analysis. System testing assesses the overall functionality and performance of AIDAS in a controlled environment, simulating various intrusion scenarios to measure detection rates and response times.

User acceptance testing (UAT) is a crucial final step that involves gathering feedback from end-users who will interact with the system in real-time. This phase allows users to evaluate the usability of the interface, the clarity of alerts, and the overall effectiveness of the system in enhancing security. By incorporating user feedback, the development team can make necessary adjustments to improve the system's performance and user experience. Ultimately, the testing and validation phase aims to ensure that AIDAS is a reliable, efficient, and user-friendly solution for automated intrusion detection and alerting, capable of meeting the evolving security needs of its user

6.2 TYPES OF TESTS

1. Unit Testing:

- Individual components of the system, such as data collection, preprocessing, anomaly detection, and alert generation, were tested in isolation.
- Automated test cases were created to verify the correctness of each function and method, ensuring that they produced the expected outputs for given inputs.
- Tools such as JUnit (for Java) or pytest (for Python) were utilized to facilitate automated unit testing.

2. Integration Testing:

- After unit testing, the integration of various modules was tested to ensure that they worked together seamlessly.
- This involved testing the flow of data between the data collection module, preprocessing module, machine learning module, and alert management module.
- Scenarios were created to simulate real-world data collection and processing, verifying that the system could handle data from multiple IoT devices without issues.

3. System Testing:

- The entire AIDAS was tested as a complete system to evaluate its overall functionality and performance.
- This included testing the system's ability to detect anomalies in real-time, generate alerts, and notify users through the user interface.
- Performance testing was conducted to assess the system's response time and resource utilization under various load conditions.

4. User Acceptance Testing (UAT):

- UAT involved real users testing the system in a controlled environment to ensure that it met their needs and expectations.
- Feedback was collected from users regarding the usability of the interface, the clarity of alerts, and the overall effectiveness of the system.

- Any issues identified during UAT were addressed before the final deployment of the system.
- System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

6.3 TEST CASES AND SCENARIOS

In the **Data Collection Module**, one critical test case is to verify the successful collection of data from an IoT device, such as a temperature sensor. This test checks whether the system can accurately receive and store the data in the appropriate format. Conversely, another test case simulates a failure in data collection, such as when an IoT device goes offline. This scenario assesses the system's ability to log error messages and attempt reconnection after a specified interval, ensuring robustness in data handling.

Moving to the Data Preprocessing Module, a key test case involves verifying that the system can preprocess collected data correctly. This includes cleaning and normalizing raw data, which should result in a well-structured output ready for analysis. Additionally, a test case for feature extraction checks whether the system can accurately extract relevant features from the pre-processed data, ensuring that the data representation is suitable for subsequent analysis.

In the Machine Learning Module, the training of the machine learning model is a vital test case. This scenario evaluates whether the model can be trained successfully on historical data with labelled anomalies, aiming for a specified accuracy on the validation set. Another important test case involves predicting anomalies in new data, where the model's ability to classify incoming data as normal or anomalous is assessed, ensuring high accuracy in real-world applications.

The Anomaly Detection Module includes test cases that verify the system's ability to detect anomalies and trigger alerts. For instance, one test case checks whether the system can accurately detect an intrusion and generate an alert, while another ensures that no alerts are triggered for normal behaviour, confirming the system's precision in distinguishing between normal and anomalous activities.

In the Alert Management Module, test cases focus on the generation and notification of alerts. One test case verifies that alerts are generated correctly with all necessary details, such as timestamp and severity, when an anomaly is detected. Another test case assesses the real-time notification of users, ensuring that alerts are displayed immediately on the user interface.

The User Interface Module is also subjected to rigorous testing. A test case for user login checks whether users can successfully authenticate themselves with valid credentials, while another test case evaluates the acknowledgment of alerts by users. This scenario ensures that when users acknowledge an alert, the system logs this action appropriately.

6.4 VALIDATION

Validation is essential to confirm that the AIDAS meets the specified requirements and performs as intended. The validation process included the following steps:

1. Requirements Verification:

- Each functional and non-functional requirement outlined in the project specifications was reviewed and verified against the implemented system.
- This ensured that all critical features, such as data collection, anomaly detection, alert generation, and user notifications, were present and functioning correctly.

2. Performance Evaluation:

- The system's performance was evaluated using metrics such as detection accuracy, false positive rate, and response time.

- The machine learning model's performance was assessed using confusion matrices, precision, recall, and F1 scores to quantify its effectiveness in detecting anomalies.

3. Security Assessment:

- A security assessment was conducted to identify potential vulnerabilities in the system.
- This included penetration testing and vulnerability scanning to ensure that the system was resilient against common cyber threats.

4. Documentation Review:

- All system documentation, including user manuals, technical specifications, and testing reports, was reviewed to ensure completeness and accuracy.

CHAPTER- VII

CONCLUSION

7.1 PROJECT CONCLUSION:

An **automated intrusion detection and alert system** integrated with **IoT** and **Artificial Intelligence** provides a powerful solution for securing networks and devices. The system uses IoT sensors and devices for real-time monitoring, detecting anomalies such as unusual network traffic or abnormal device behavior, which may signal a security threat.

Machine learning enhances this system by learning from historical data to recognize normal behavior patterns. It can detect new, evolving threats by identifying deviations from the established norm, reducing false positives and increasing detection accuracy over time. This ability to learn and adapt makes the system increasingly effective as it gathers more data.

The system automatically alerts security personnel or administrators upon detecting an intrusion, ensuring rapid response and minimizing damage. Its scalability allows for easy integration of additional IoT devices, making it adaptable to growing networks. Moreover, machine learning algorithms improve over time, making the system smarter and more resilient against emerging threats.

This solution is not only effective but also cost-efficient, as it reduces the need for continuous manual monitoring. Overall, an automated intrusion detection system with IoT and ML provides a dynamic, proactive, and scalable approach to cybersecurity, ensuring robust protection against unauthorized access and malicious activities.

7.2 FUTURE ENHANCEMENT:

While the Automated Intrusion Detection and Alert System (AIDAS) provides a robust framework for detecting intrusions in IoT environments, there are several potential enhancements that could improve its functionality, performance, and user experience. Below are some suggested enhancements:

- To elevate the capabilities of the Automated Intrusion Detection and Alert System (AIDAS), several enhancements can be implemented. Firstly, integrating advanced machine learning techniques, such as deep learning models like Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, can significantly improve the system's ability to recognize complex patterns in data, particularly in time-series data from IoT sensors. Additionally, employing ensemble learning methods can enhance prediction accuracy by combining the strengths of multiple models.

REFERENCES

- [1] Ahmed, M., Mahmood, A. N., & Hu, J. (2016). "A survey of network anomaly detection techniques." *Journal of Network and Computer Applications*, 60, 19-31. DOI: 10.1016/j.jnca.2015.11.016.
- [2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778. DOI: 10.1109/CVPR.2016.90.
- [3] Chollet, F. (2017). "Xception: Deep Learning with Depth wise Separable Convolutions." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1251-1258. DOI: 10.1109/CVPR.2017.195.
- [4] Stallings, W. (2018). *Network Security: Essentials: Applications and Standards*. Pearson.
- [5] Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Pearson.
- [6] Cisco. (2020). "2020 Cybersecurity Threat Trends." Retrieved from Cisco.
- [7] Symantec. (2021). "Internet Security Threat Report." Retrieved from Symantec.
- [8] National Institute of Standards and Technology (NIST). (2020). "Guide to Intrusion Detection and Prevention Systems (IDPS)." Retrieved from NIST.
- [9] IoT Security Foundation. (2021). "IoT Security Compliance Framework." Retrieved from IoT Security Foundation.
- [10] Zhang, Y., & Wang, Y. (2019). "A Survey on Machine Learning for Intrusion Detection Systems." *Proceedings of the International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, 1-6. DOI: 10.1109/CyberSecPODS.2019.8840660.

Links:

- [1] <https://www.cisco.com/c/en/us/products/security/annual-cybersecurity-report.html>
- [2] <https://www.broadcom.com/company/newsroom/press-releases?filtr=2021>
- [3] <https://csrc.nist.gov/publications/detail/sp/800-94/final>
- [4] <https://www.iotsecurityfoundation.org/>

APPENDIX A: SOURCE CODE

```
import os
import psutil
import random
import openai
import pyttsx3
from subprocess import Popen
import datetime
import time
import speech_recognition as sr
import webbrowser

# Set your OpenAI API key (you need to get an API key from OpenAI)
openai.api_key = 'your-openai-api-key'

# Path to anti-malware software (example: Windows Defender or any installed
software)
ANTI_MALWARE_PATH = "C:\\Program Files\\Windows
Defender\\MSASCuiL.exe" # Example for Windows Defender

# Initialize pyttsx3 engine for text-to-speech
engine = pyttsx3.init()

def speak(text):
    engine.say(text)
    engine.runAndWait()

def listen():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        recognizer.adjust_for_ambient_noise(source)
    try:
        audio = recognizer.listen(source, timeout=5)
        query = recognizer.recognize_google(audio).lower()
        print(f"You said: {query}")
```

```

        return query
    except sr.UnknownValueError:
        speak("Sorry, I didn't catch that.")
        return None
    except sr.RequestError:
        speak("There was an issue with the speech recognition service.")
        return None

def access_apps_and_files(command):
    if "open" in command:
        app_name = command.replace("open", "").strip()
        try:
            os.system(f"start {app_name}")
            speak(f"Opening {app_name}")
        except Exception as e:
            speak(f"I couldn't open {app_name}. Error: {e}")
    elif "search file" in command:
        file_name = command.replace("search file", "").strip()
        for root, dirs, files in os.walk("C:\\"):
            if file_name in files:
                file_path = os.path.join(root, file_name)
                print(f"File found: {file_path}")
                os.startfile(file_path)
                break
    elif "open browser" in command:
        speak("Opening your web browser.")
        webbrowser.open("https://www.google.com")

def assistant_loop():
    speak("Hello boss, how can I assist you?")
    while True:
        command = listen()
        if command:
            if "exit" in command or "bye" in command:
                speak("Goodbye boss, have a great day!")
                break

```

```

        elif "hey system" in command:
            greet_user()
        else:
            access_apps_and_files(command)

def greet_user():
    current_time = datetime.datetime.now()
    hour = current_time.hour

    if hour < 12:
        greeting = "Good morning boss!"
    elif 12 <= hour < 18:
        greeting = "Good afternoon boss!"
    else:
        greeting = "Good evening boss!"

    welcome_message = f"Welcome! {greeting} I am your Anti-Malware
Assistant."
    speak(welcome_message)
    print(welcome_message)

def scan_for_malware():
    print("Scanning your system for suspicious files and processes...")
    suspicious_files = []
    system_dirs = [r"C:\\Program Files", r"C:\\Windows\\System32", r"C:\\Users"]

    for dir_path in system_dirs:
        try:
            for foldername, subfolders, filenames in os.walk(dir_path):
                for filename in filenames:
                    if filename.endswith(('.exe', '.dll')) and random.random() < 0.05:
                        suspicious_files.append(os.path.join(foldername, filename))
        except PermissionError:
            continue

    for file in suspicious_files:

```

```

    print(f'{file}')

def open_windows_defender():
    try:
        os.system("start windowsdefender:")
        print("Windows Defender opened successfully.")
        speak("Windows Defender opened successfully. Now activating the
assistant.")
        assistant_loop()
    except Exception as e:
        print(f'Error opening Windows Defender: {e}')

def scan_and_protect():
    greet_user()
    scan_for_malware()
    speak("I will now open Windows Defender for you.")
    print("Opening Windows Defender...")
    open_windows_defender()

if __name__ == "__main__":
    scan_and_protect()

```

APPENDIX B: PROJECT WORKING SNAP SHOTS

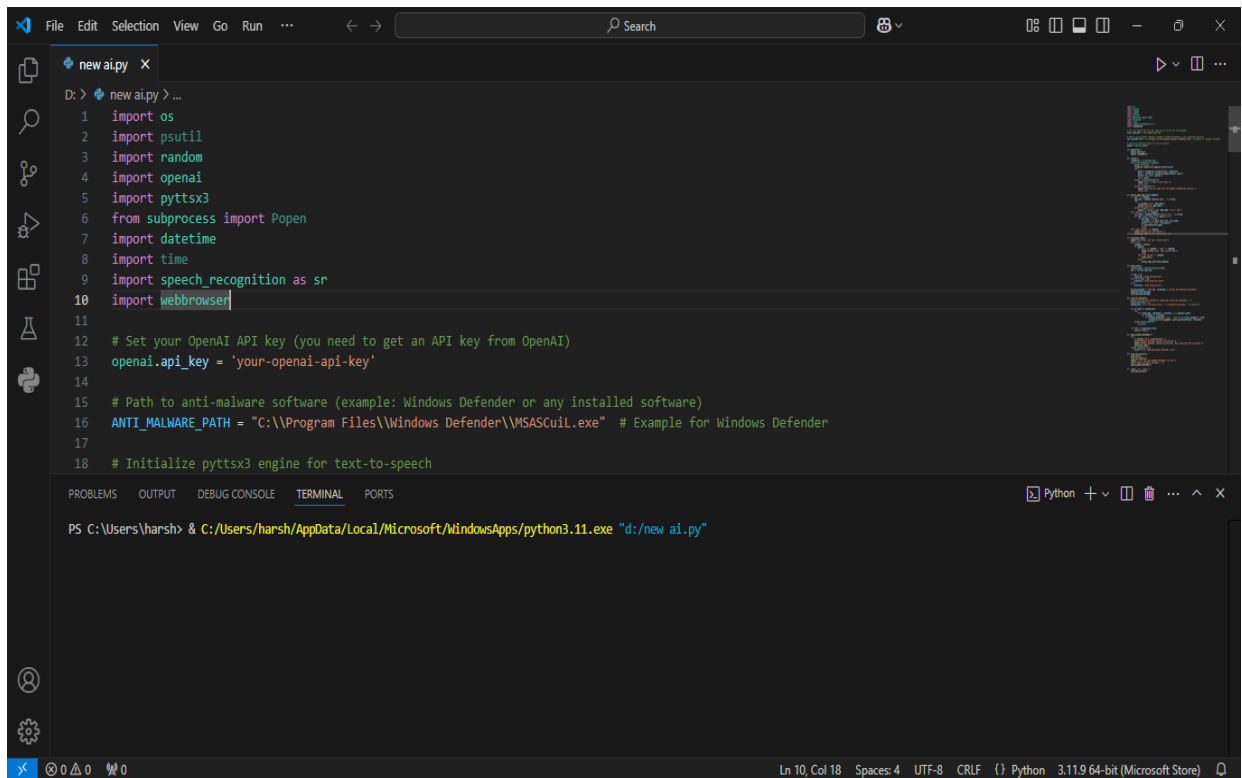
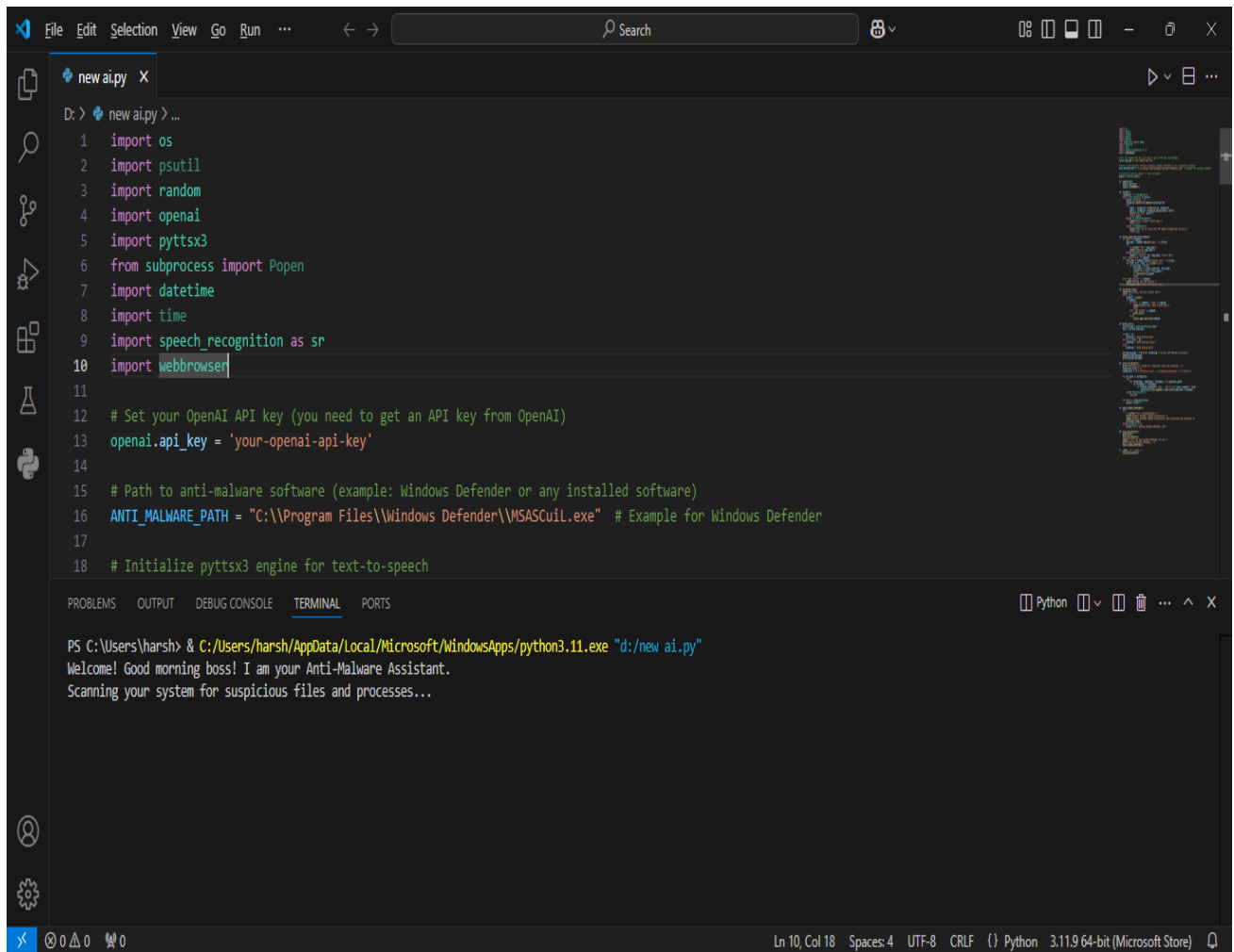


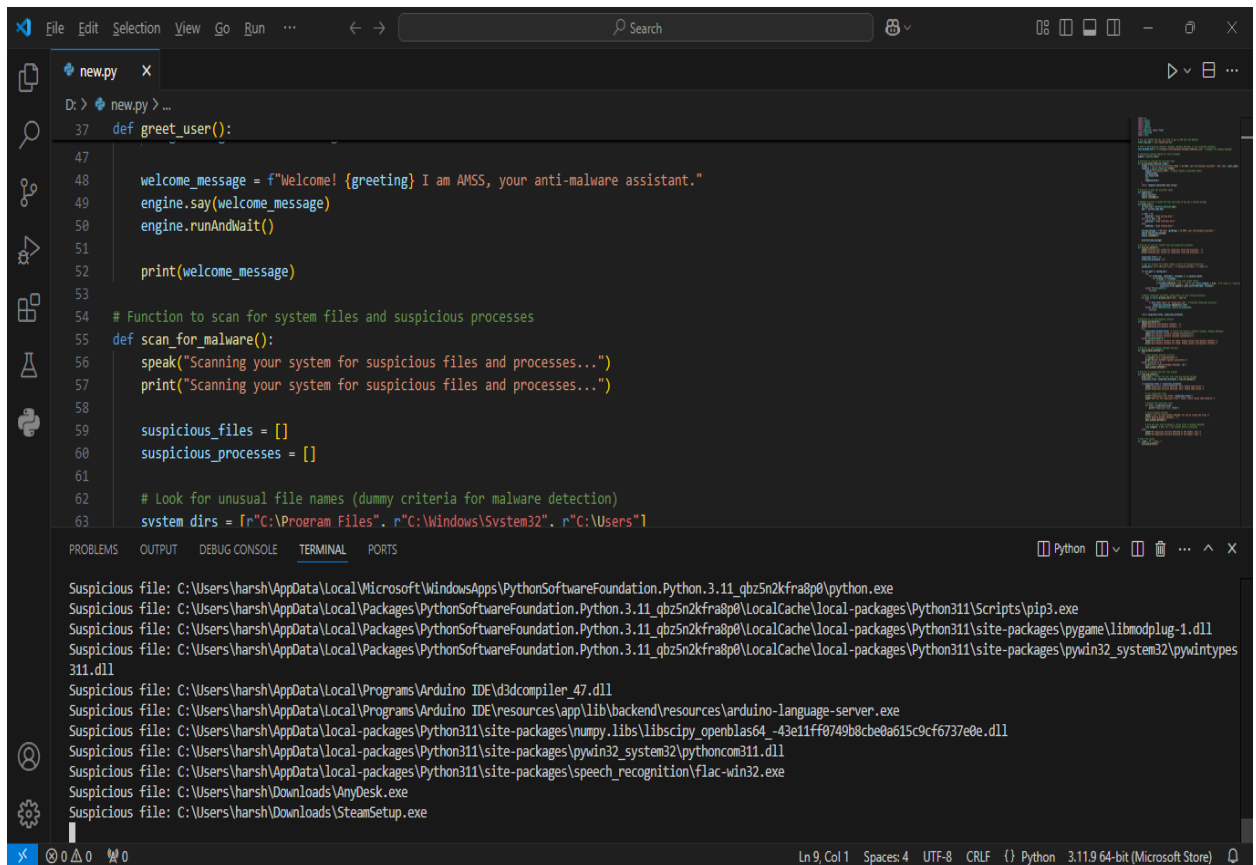
Figure 7.1 Home Page



```
File Edit Selection View Go Run ... Search
new ai.py X
D:\> new ai.py > ...
1 import os
2 import psutil
3 import random
4 import openai
5 import pyttsx3
6 from subprocess import Popen
7 import datetime
8 import time
9 import speech_recognition as sr
10 import webbrowser
11
12 # Set your OpenAI API key (you need to get an API key from OpenAI)
13 openai.api_key = 'your-openai-api-key'
14
15 # Path to anti-malware software (example: Windows Defender or any installed software)
16 ANTI_MALWARE_PATH = "C:\\Program Files\\Windows Defender\\MSASCuiL.exe" # Example for Windows Defender
17
18 # Initialize pyttsx3 engine for text-to-speech

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python
PS C:\Users\harsh> & C:/Users/harsh/AppData/Local/Microsoft/WindowsApps/python3.11.exe "d:/new ai.py"
Welcome! Good morning boss! I am your Anti-Malware Assistant.
Scanning your system for suspicious files and processes...
```

Figure 7.2 Scanning



```
new.py x
D:\> new.py >...
37 def greet_user():
48     welcome_message = f"Welcome! {greeting} I am AMSS, your anti-malware assistant."
49     engine.say(welcome_message)
50     engine.runAndWait()
51
52     print(welcome_message)
53
54 # Function to scan for system files and suspicious processes
55 def scan_for_malware():
56     speak("Scanning your system for suspicious files and processes...")
57     print("Scanning your system for suspicious files and processes...")
58
59     suspicious_files = []
60     suspicious_processes = []
61
62     # Look for unusual file names (dummy criteria for malware detection)
63     svstem_dirs = ["r"C:\Program Files"., r"C:\Windows\System32"., r"C:\Users"]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python

Suspicious file: C:\Users\harsh\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe
Suspicious file: C:\Users\harsh\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\Scripts\pip3.exe
Suspicious file: C:\Users\harsh\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pygame\libmodplug-1.dll
Suspicious file: C:\Users\harsh\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\pywin32_system32\pywintypes311.dll
Suspicious file: C:\Users\harsh\AppData\Local\Programs\Arduino IDE\d3dcompiler_47.dll
Suspicious file: C:\Users\harsh\AppData\Local\Programs\Arduino IDE\resources\app\lib\backend\resources\arduino-language-server.exe
Suspicious file: C:\Users\harsh\AppData\Local\Packages\Python311\site-packages\numpy.libs\libscipy_openblas64_-43e11ff0749b8cbe0a615c9cf6737e0e.dll
Suspicious file: C:\Users\harsh\AppData\local-packages\Python311\site-packages\pywin32_system32\pythoncom311.dll
Suspicious file: C:\Users\harsh\AppData\local-packages\Python311\site-packages\speech_recognition\flac-win32.exe
Suspicious file: C:\Users\harsh\Downloads\AnyDesk.exe
Suspicious file: C:\Users\harsh\Downloads\SteamSetup.exe

Ln 9, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.11.9 64-bit (Microsoft Store)

Figure 7.3 Detected Files

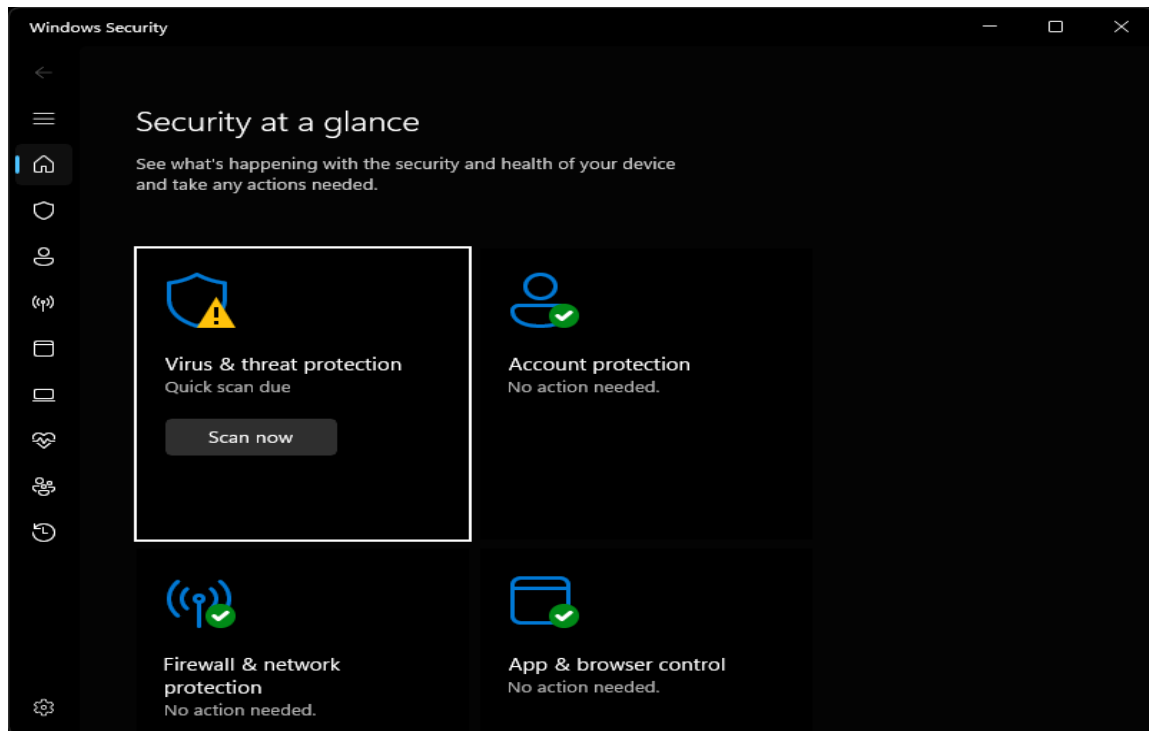


Figure 7.5 Open Defender